

Creating functions in R - Handout

1 Exercise 1

Write a function (name it `mysummary`) to compute and display the average and variance of a (vector) sample `x`, using functions such as `mean()`, `var()`, `return()` and `list()`¹.

What follows is a proposition of a function that computes the wanted quantities. This function will return a `list` containing the results of the computations using the basic functions.

```
mysummary <- function(x) {  
  res <- list(mean(x), var(x), range(x))  
  names(res) <- c('mean of x', 'variance of x', 'min and max of x')  
  return(res)  
}
```

In order to test our function, we shall create a vector of 100 observations drawn from a Standard Normal distribution on which, we will apply our function. We can store the results of this operation in an object named `my.sum` and retrieve its elements using the operator `$` and their respective names.

```
x <- rnorm(100)  
  
my.sum = mysummary(x)  
  
my.sum[[1]]  
  
## [1] 0.1642238  
  
my.sum$'mean of x'
```

¹Hint: you can look at R built-in functions as examples, by typing the name of the function in the console. Compare `median` and `mean`.

```
## [1] 0.1642238

mysummary(x)$'mean of x' # since the result of our function is a list.

## [1] 0.1642238

mysummary(x)[[2]] # alternative way of retrieving the second element.

## [1] 0.8144796
```

2 Exercise 2

The goal of this exercise is to find numerical solutions of a set of estimating (score) equations and to compare them with those obtained with the R built-in functions.

Let's consider the `breast-cancer-wisconsin.txt` dataset. We will use the response status (coded 0/1) and only one explanatory variable `nuclei` (plus an intercept).

We consider the logit function as the link between the random part $\mathbb{E}[Y_i|\mathbf{x}_i]$ and the systematic part (linear predictor) $\eta_i = \beta_0 + \beta_1 x_{i1}$.

Based on this, the formula for Newton-Raphson algorithm is:

$$\beta^{(k)} = \beta^{(k-1)} + \left\{ \sum_{i=1}^n \frac{\exp(-\mathbf{x}_i^T \beta^{(k-1)})}{[1 + \exp(-\mathbf{x}_i^T \beta^{(k-1)})]^2} \mathbf{x}_i \mathbf{x}_i^T \right\}^{-1} \times \left\{ \sum_{i=1}^n \left[y_i - \frac{1}{1 + \exp(-\mathbf{x}_i^T \beta^{(k-1)})} \right] \mathbf{x}_i \right\}.$$

- a) Write an R function that computes an update of the estimated parameters, given the previous value, according to the Newton-Raphson algorithm.

We build a function that computes `Ubeta`, which is $U(\beta, y)$ and `Jbeta` which is the Fisher information matrix.

```
updatebeta <- function(beta, y, X) {
  eta <- as.vector(X %*% beta)
  Ubeta <- t(X) %*% (y - exp(eta) / (1 + exp(eta)))
  Jbeta <- t(X) %*% diag(exp(eta) / (1 + exp(eta))^2) %*% X
  beta + solve(Jbeta, Ubeta)
}
```

- b) Taking the null vector as the starting value for β , perform a few iterations of the Newton-Raphson procedure using a `while()` in the following way:

```
while (max(abs(beta0-beta1))>tol & it<maxit){
  ...
}
```

where `beta0` and `beta1` are the actual and next beta estimations, `tol` is equal to 10^{-8} and `maxit` is 50. Report the estimated β in a table.

We first have to think about changing the status variable from (4/2) to (1/0).

```
breast <- read.table("breast-cancer-wiconsin.txt", header=TRUE)

breast$status[which(breast$status == 4)] <- 1
breast$status[which(breast$status == 2)] <- 0
```

Then we initialize our variables:

```
y <- na.omit(breast)$status # responses in the model
x <- cbind(rep(1, length(y)), na.omit(breast)$nuclei)
#Construct the design matrix for our model

tol <- 1e-8 # we set tol to an arbitrary value
maxit <- 50 # same for maxit
beta0 <- c(0, 0) #initialize the beta, could be set to any values
```

See R code lines 67-74 for the `while()` loop.

- c) Write an R function that computes the log-likelihood of your model.

Using the loglikelihood expression of the Bernoulli distribution, be attentive to the fact that we introduce a minus sign because `optim()` minimizes by default

```
loglikelihood <- function(beta, x, y) {  
  n <- nrow(x)  
  loglike <- 0  
  for (i in 1:n) {  
    etai <- -x[i, ] %*% beta  
    loglike <- loglike + (1 - y[i]) * etai - log(1 + exp(etai))  
  }  
  return(-loglike) # optim minimizes - we need to put a minus  
}
```

- d) Use the built-in function `optim()` to solve the maximization problem (with the “BFGS” method). Compare your results with those in b).

See lines 96-102 for details.

3 Exercise 3

Consider the profile likelihood of page 112 of the course slides²:

$$L_1(\theta) = \max_{\delta} L(\theta, \delta),$$

where θ is the parameter of interest of dimension 1, and δ is the vector of the other model parameters. Under this formulation, we construct the following Likelihood Ratio (LR) statistic to test $H_0 : \theta = \theta_0$:

$$\text{LR}(\theta_0) = 2 \left[\log L(\hat{\theta}, \hat{\delta}) - \log L(\theta_0, \hat{\delta}) \right] = 2 \left[\log L_1(\hat{\theta}) - \log L_1(\theta_0) \right] \underset{H_0}{\sim} \chi_1^2,$$

where $\hat{\theta}, \hat{\delta}$ are the maximum likelihood (ML) estimators of θ and δ . The non-rejection region of this test is determined by the condition $\text{LR}(\theta_0) < F^{-1}(1 - \alpha)$, where F is the cdf of the χ_1^2 distribution, or equivalently, all the θ_0 such that:

$$\log L_1(\theta_0) > \log L(\hat{\theta}, \hat{\delta}) - \frac{1}{2} F^{-1}(1 - \alpha) \quad (1)$$

²Based on a handout by Professor David Patterson, University of Montana <http://www.math.umt.edu/patterson/>

The points within the *Profile Likelihood* CI will be those θ_0 within the non-rejection region of the LR test.

Consider the goose permit data (file `goose.txt`) from Bishop and Heberlein (*Amer. J. Agr. Econ.* 61, 1979), where 237 hunters were each offered one of 11 cash amount (bids) ranging from 1\$ to 200\$ in return of their permits. We use a logit model with $\log(\text{bid})$ as the explanatory variable (in addition to the intercept):

$$\log\left(\frac{\pi_i}{1 - \pi_i}\right) = \beta_0 + \beta_1 \log(\text{bid}_i)$$

where π_i is the probability that a permit holder would accept bid amount bid_i .

- a) Estimate the parameters of the model, extract the log-likelihood with `logLik()` to determine the cutoff value as in (1) for $\alpha = 0.05$ and $\alpha = 0.01$. Compute the profiled likelihood CI for β_1 with the `confint()` function.

As usual, we use the `glm()` function to obtain estimates of the model parameters.

When package MASS is loaded, the function `confint()` provides the desired PL confidence intervals, whereas the `confint.default()` yields the typical Normal (asymptotic) confidence intervals.

```
goose.plci.05 = confint(goose.glm, level=.95)
## Waiting for profiling to be done...

goose.nlci.05 = confint.default(object=goose.glm)
cbind(goose.plci.05, goose.nlci.05)

##           2.5 %    97.5 %           2.5 %    97.5 %
## (Intercept) -5.783372 -3.317595 -5.6807434 -3.224353
## log(bid)      0.978029  1.667461  0.9530795  1.639647

goose.plci.01 = confint(goose.glm, level=.99)
## Waiting for profiling to be done...

goose.nlci.01 = confint.default(object=goose.glm, level=0.99)
cbind(goose.plci.01, goose.nlci.01)

##           0.5 %    99.5 %           0.5 %    99.5 %
## (Intercept) -6.2463329 -2.996867 -6.066670 -2.838426
## log(bid)      0.8875305  1.796307  0.845212  1.747514
```

b) In the following points, we intend to construct PL Confidence Intervals *manually*.

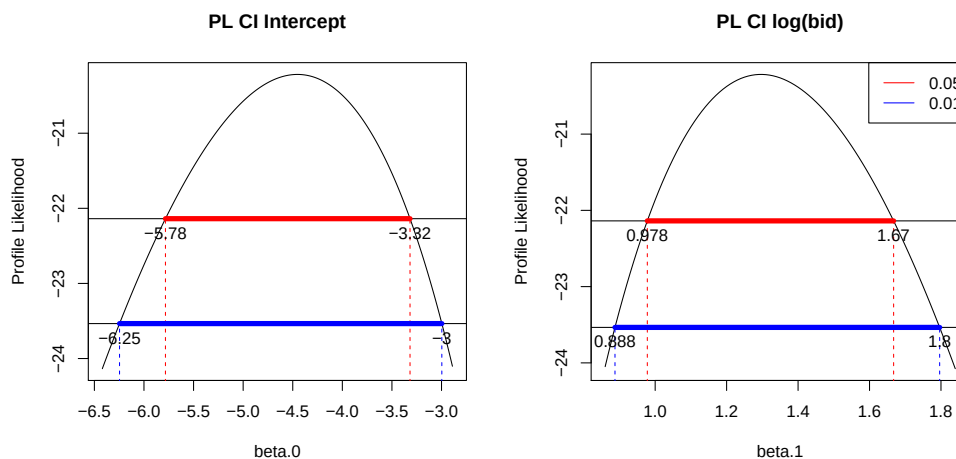
- (a) Write an R code to compute the profile likelihood function $L_1(\beta_1)$ (which make use of the estimates of the previous point).

Since the Profile Likelihood function is the maximization of the Likelihood with respect to one of the parameters while fixing the value of the other ones, we could think of using the `offset=...` argument in the `glm()` function. We could thus optimize the likelihood with respect to, say β_0 , while fixing the value of β_1 by typing `offset =  $\beta_1$` and redefining the formula so that it depends only on β_0 .

Moreover, we could consider performing this optimization on a grid of values for β_1 , and thus obtain values of the profile likelihood for this grid. The resulting values of the log-likelihood (extracted via `logLik()`) are thus the evaluations of the Profile Likelihood. (See the code).

- (b) Plot the profile likelihood $L_1(\beta_1)$ and the cutoff values.

In the following plot, we draw the PL function for both parameters. The values of the PL confidence intervals will be those in the grid whose evaluations are above the cutoff values, depending on the optimized likelihood and quantiles of the chi-squared distribution. For this illustration, we have considered CI of levels 95% and 99%



- (c) Compare the results with the CI computed at point a).

As we have said before, we can evaluate points in the grid and obtain the limits of the PLCI, by selecting the minimal (maximal) points in the grid such that their PL evaluations are beyond the cutoff rule. It is apparent that these values bare resemblance to the ones obtained via the `confint()` method (improvements can be made by improving the grid).

We compare obtained values for β_0

```
##                2.5 %    97.5 %  
## Intercept.plci.95 -5.783372 -3.317595  
## Intercept.manl.95 -5.780000 -3.320000  
##                0.5 %    99.5 %  
## Intercept.plci.99 -6.246333 -2.996867  
## Intercept.manl.99 -6.240000 -3.000000
```

We compare obtained values for β_1

```
##                2.5 %    97.5 %  
## log_bid.plci.95 0.978029 1.667461  
## log_bid.manl.95 0.980000 1.660000  
##                0.5 %    99.5 %  
## log_bid.plci.99 0.8875305 1.796307  
## log_bid.manl.99 0.8900000 1.790000
```